SRM Souss-Massa

Gestion de flotte et de logistique

Guide technique

Manuel du developpeur SRM Fleet

Comprendre, faire evoluer et maintenir l'application

Redige par

Oussama BENAZZOUZ • **TAKI Fadlouallah**

27 août 2026

Table des matières

1 Introduction **1**

2 Vue d'ensemble de l'architecture **2**
2.1 Deux roles, un cloisonnement . . . 2

3 Le backend **2**
3.1 Une organisation par module . . . 2
3.2 Le parcours d'une requete . . . 3
3.2.1 1. Le controleur : mince, il ne fait que router . . . 3
3.2.2 2. Le service : les regles metier et la transaction . . . 3
3.2.3 3. Repository, entite et DTO . . . 3
3.3 Les memes couches partout . . . 3
3.4 La securite . . . 4
3.5 Le cloisonnement par province (RG14) . . . 4
3.6 Les filtres de recherche (Specifications) . . . 4
3.7 Validation et gestion des erreurs . . . 4
3.8 Les cycles de vie . . . 5
3.9 Le moteur d'alertes . . . 5
3.10 Base de donnees et migrations . . . 5
3.11 Les exports CSV et PDF . . . 5

4 Le frontend **5**
4.1 Organisation . . . 5
4.2 Le parcours d'un module (cote client) . . . 6
4.3 L'authentification cote client . . . 6
4.4 Le design system maison . . . 6

5 Les regles de gestion cles **6**

6 Ajouter un nouveau module **7**
6.1 Cote backend . . . 7
6.2 Cote frontend . . . 7

7 Conventions Git et flux de travail **7**
7.1 Branches . . . 7
7.2 Commits (Conventional Commits) . . . 7
7.3 Pull requests . . . 8
7.4 Regles de code . . . 8

8 Decisions d'architecture (resume) **8**

9 Tests et integration continue **8**

1 Introduction

SRM Fleet est le backoffice web de gestion du parc automobile de SRM Souss-Massa. Il couvre le referentiel des vehicules et des conducteurs, le cycle de vie des affectations, les cartes

carburant, la conformite reglementaire (assurance, visite technique, sinistres), un moteur d'alertes, un tableau de bord et des exports CSV et PDF.

Ce guide s'adresse a toute personne qui reprendra le code. Il explique l'architecture, le fonctionnement d'un module de bout en bout, les briques communes a tous les modules, puis la facon d'ajouter un module en respectant les conventions du projet. Il reste volontairement synthetique : un seul module est detaille, les autres suivent le meme schema.

Couche	Technologie
Frontend	Angular 21 (composants standalone, signaux), design system maison, servi par Nginx.
Backend	Spring Boot 3.5 sur Java 21, Spring Security (JWT), Spring Data JPA.
Base de donnees	PostgreSQL 17, migrations Flyway.
Generation PDF	OpenPDF.
Orchestration	Docker Compose.

2 Vue d'ensemble de l'architecture

L'application suit une architecture classique en trois tiers, sans etat cote serveur.

> **Navigateur** (Angular) → **Nginx** sert l'application et relaie `/api/` → **API REST Spring Boot** → **PostgreSQL**.
> L'authentification repose sur un jeton **JWT** envoye dans l'en-tete `Authorization`. Le backend est **stateless** : aucune session serveur, chaque requete porte son jeton.

2.1 Deux roles, un cloisonnement

- **Administrateur central** : visibilite nationale, gere le referentiel et valide les demandes de carte.
- **Responsable provincial** : acces limite a sa province, sans visibilite sur les autres.

Ce cloisonnement (regle **RG14**) est applique dans la couche service du backend. Il est present dans presque tous les modules : c'est un reflexe a garder en tete pour toute nouvelle fonctionnalite.

3 Le backend

3.1 Une organisation par module

Le code est range **par domaine metier**, pas par type technique. Chaque module contient sa propre pile de classes, ce qui rend chaque fonctionnalite autonome et facile a retrouver. Exemple pour le module Vehicule :

```
ma/srm/fleet/referentiel/vehicule/
  VehiculeController.java       # exposition REST (couche mince)
  VehiculeService.java          # regles metier + transactions
  VehiculeRepository.java       # acces base (Spring Data JPA)
  VehiculeSpecifications.java   # filtres de recherche dynamiques
  Vehicule.java                 # entite JPA (table)
  StatutVehicule.java           # enumerations du domaine
  dto/VehiculeRequest.java      # donnees entrantes (validees)
  dto/VehiculeResponse.java     # donnees sortantes (jamais l'entite)
```

3.2 Le parcours d'une requete

Toutes les requetes suivent le meme trajet en couches. On le detaille ici sur la creation d'un vehicule ; **tous les modules fonctionnent ainsi**.

1. Le controleur : mince, il ne fait que router

Il declare la route, controle le role si besoin, et delegue au service. Aucune logique metier ici.

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
@PreAuthorize("hasRole('ADMINISTRATEUR_CENTRAL')")
public VehiculeResponse creer(@Valid @RequestBody VehiculeRequest req) {
    return service.creer(req);
}
```

Points a retenir : `@Valid` declenche la validation des champs, `@PreAuthorize` restreint l'acces au role, `@AuthenticationPrincipal UserPrincipal user` (sur les lectures) fournit l'utilisateur connecte pour le cloisonnement.

2. Le service : les regles metier et la transaction

Annote `@Transactional`, il porte toute la logique : unicite, regles de gestion, resolution des references, puis conversion en DTO.

```
public VehiculeResponse creer(VehiculeRequest req) {
    if (vehiculeRepository.existsByImmatriculation(req.immatriculation())) {
        throw new BusinessException("Cette immatriculation existe deja",
    HttpStatus.CONFLICT);
    }
    Vehicule v = new Vehicule();
    appliquer(v, req);                          // RG2 : location => fournisseur
    obligatoire
    return toResponse(vehiculeRepository.save(v));
}
```

3. Repository, entite et DTO

Le **repository** etend `JpaRepository` : les methodes de base (save, findById, findAll) sont fournies ; on ajoute des methodes derivees du nom (`existsByImmatriculation`). L'**entite** represente la table. Le **DTO** (un `record` Java) transporte les donnees : l'entite JPA ne quitte jamais le service, le controleur ne parle qu'en DTO.

> Regle d'or du projet : **injection par constructeur**, **aucune logique metier dans les controleurs**, **les entites ne sortent pas de la couche service**. Le mapping entite vers DTO est ecrit a la main (methode `toResponse` ou classe `...Mapper`), sans librairie.

3.3 Les memes couches partout

Chaque module reprend ce schema. La liste ci-dessous situe les principaux modules ; leur structure interne est identique a celle du module Vehicule.

Module	Role
Referentiel	Vehicules, conducteurs et permis, fournisseurs, provinces, departements.
Affectation	Lien vehicule vers conducteur, historique, cloture.
Carte carburant	Demande, validation par le central, emission de la carte.
Conformite	Assurances, visites techniques, sinistres et pieces jointes.
Alerte	Moteur planifie et configuration des seuils.
Dashboard	Agregation des statistiques de tous les modules.
Export	Generation CSV et PDF, reutilisant les filtres des listes.
Admin et Auth	Comptes utilisateurs, connexion et jetons JWT.

3.4 La securite

La configuration est centralisee dans `SecurityConfig`. Choix structurants : **CSRF desactive** (inutile pour une API a jeton), **sessions STATELESS**, et une **liste blanche** d'endpoints publics (login, healthcheck, documentation Swagger). Tout le reste exige une authentification (principe du moindre privilege).

- **Connexion** : `/api/v1/auth/login` verifie les identifiants via l'`AuthenticationManager`, puis `JwtService` genere le jeton.
- **Chaque requete suivante** : le `JwtAuthenticationFilter` lit le jeton, le valide et peuple le contexte de securite avant les filtres standards de Spring.
- **Roles** : `@PreAuthorize("hasRole('ADMINISTRATEUR_CENTRAL')")` protege les actions reservees au central (creation de referentiel, validation des demandes).
- **Mots de passe** : haches avec **BCrypt** (sel integre).

3.5 Le cloisonnement par province (RG14)

Dans les services, un Responsable provincial ne voit que sa province. En **lecture de liste**, on ajoute une specification qui filtre sur `provinceId`. En **acces a un element precis**, on verifie la portee ; si l'element appartient a une autre province, on renvoie **404** (et non 403) pour ne pas reveler son existence.

```
if (user.getRole() == RoleUtilisateur.RESPONSABLE_PROVINCIAL
        && !v.getProvince().getId().equals(user.getProvinceId())) {
    throw new BusinessException("Vehicule introuvable : " + v.getId(),
    HttpStatus.NOT_FOUND);
}
```

3.6 Les filtres de recherche (Specifications)

Les listes acceptent des filtres optionnels (immatriculation, type, statut, province). On les compose avec l'API `Specification` de Spring Data : chaque critere est une brique, ignoree si le parametre est absent. Le cloisonnement province se branche comme une brique de plus.

3.7 Validation et gestion des erreurs

Deux niveaux, une reponse homogene :

- **Validation des champs** : annotations Bean Validation sur les DTO (`@NotBlank`, `@NotNull`...), declenchees par `@Valid`.
- **Regles metier** : on leve une `BusinessException` portant un message et un statut HTTP.

Le `GlobalExceptionHandler` (annote `@RestControllerAdvice`) transforme toute exception en reponse **ProblemDetail** (norme RFC 7807 : un corps JSON standard avec titre, statut, detail). Les erreurs inattendues renvoient un 500 generique, sans exposer de detail technique.

3.8 Les cycles de vie

Plusieurs modules portent une petite machine a etats, geree dans le service (transitions verifiees, statut stocke en base) :

- **Demande de carte** : `EN_ATTENTE` puis `VALIDEE` ou `REFUSEE`. La validation (par le central) fixe le plafond et le fournisseur, et emet la carte.
- **Affectation** : `ACTIVE` puis `CLOTUREE`. La cloture conserve l'historique.
- **Sinistre** : `OUVERT`, `EN_COURS`, `CLOTURE`, avec pieces jointes.

3.9 Le moteur d'alertes

Un service planifie (Spring Scheduler, une fois par jour) parcourt les configurations de seuils. Pour chaque type surveille (assurance, visite technique), il retient les echeances qui tombent dans la fenetre du seuil et cree une alerte **si elle n'existe pas deja** (deduplication par type et reference). Les notifications sont **in-app** uniquement (canal tableau de bord) ; le tableau de bord des alertes les classe par urgence.

3.10 Base de donnees et migrations

Le schema est gere par **Flyway** : des fichiers `V1__...sql` a `V8__...sql` appliques automatiquement au demarrage, dans l'ordre. On ne modifie jamais une migration deja livree : on en ajoute une nouvelle.

> Deux regles critiques reposent sur des **index uniques partiels PostgreSQL**, impossibles a reproduire avec une base en memoire : **RG3** (une seule affectation active par vehicule) et **RG4** (une seule carte active par affectation). C'est la base qui garantit ces regles, en plus du controle applicatif. Elles sont verifiees par des tests d'integration Testcontainers.

Le **seed** initial insere le referentiel de base et le compte `admin` (dont le mot de passe est pose au demarrage par `AdminPasswordInitializer`, a partir de la configuration : aucun mot de passe par defaut, voir le guide de deploiement).

3.11 Les exports CSV et PDF

Chaque liste propose un bouton d'export. Les exports **reutilisent les services de liste** : memes filtres et meme cloisonnement province que l'ecran. Le **CSV** est genere sans dependance (UTF-8 avec BOM pour Excel, separateur point-virgule). Le **PDF** (OpenPDF) est mis en page selon la charte graphique : logo, filet tricolore officiel, tableau a en-tete bleu, pied de page.

4 Le frontend

4.1 Organisation

L'application Angular est decoupee en trois espaces :

- `core/` : services transverses (authentification, theme, composants d'interface communs).
- `features/` : un dossier par module (vehicules, conducteurs, affectations...), avec ses composants, son service HTTP, ses modeles et ses routes.
- `layout/` : la coquille de l'application (menu lateral, barre superieure).

Tous les composants sont **standalone** (pas de NgModule), l'etat local s'exprime avec des **signaux**, et le typage est strict (pas de `any`).

4.2 Le parcours d'un module (cote client)

Meme regularite qu'au backend. Pour un module donne :

- un **service** expose les appels HTTP vers `/api/v1/...` et renvoie des `Observable` ;
- trois **composants** type : la **liste** (recherche et filtres), le **formulaire** (creation et modification), le **detail** (fiche en lecture) ;
- les **modeles** (interfaces TypeScript) decrivent les donnees echangees.

```
rechercherVehicules(f: RechercheVehicule): Observable<Vehicule[]> {
  let params = new HttpParams();
  if (f.immatriculation) params = params.set('immatriculation', f.immatriculation);
  if (f.statut) params = params.set('statut', f.statut);
  return this.http.get<Vehicule[]>('/api/v1/vehicules', { params });
}
```

4.3 L'authentification cote client

Le `AuthService` est la source de verite de la session. Il conserve le jeton et le profil dans le `localStorage` et expose des signaux (`isAuthenticated`, `isAdmin`). Deux mecanismes completent le tableau :

- **Intercepteurs HTTP** : `authInterceptor` ajoute l'en-tete `Authorization` sur les appels `/api/` ; `errorInterceptor` traite les erreurs (deconnexion sur 401).
- **Gardes de routes** : `authGuard` (routes authentifiees), `guestGuard` (page de connexion), `adminGuard` (routes reservees au central).

La session est restauree de facon synchrone au demarrage, avant toute decision de routage. La validite du jeton (champ `exp`) est verifiee cote client par confort ; la validation qui fait foi reste celle du backend.

4.4 Le design system maison

Aucune librairie d'interface externe. Tout part des **jetons** (couleurs, espacements, rayons, typographie) et des **primitives** CSS definies dans `frontend/src/styles.scss` : `.card`, `.btn`, `.field`, `.input`, `.table`, `.badge`... Chaque nouvel ecran se compose a partir de ces primitives, ce qui garantit une coherence visuelle totale et un theme clair et sombre gere en un seul endroit. Les graphes du tableau de bord sont dessines en SVG.

5 Les regles de gestion cles

Ces regles structurent le metier. Elles sont appliquees dans les services et, pour les plus critiques, garanties par la base.

Regle	Enonce
RG2	Un vehicule en location doit referencer un fournisseur.
RG3	Un vehicule ne peut avoir qu'une seule affectation active a la fois (index unique partiel).
RG4	Une affectation ne peut avoir qu'une seule carte carburant active (index unique partiel).
RG11	Le conducteur accompagnateur d'une visite est deduit de l'affectation active a la date.
RG14	Un Responsable provincial n'accede qu'aux donnees de sa province.

6 Ajouter un nouveau module

La regularite du projet rend l'ajout d'un module mecanique. Voici la recette, en reprenant le module Vehicule comme modele.

6.1 Cote backend

1. Creer le package `ma/srm/fleet/<domaine>/<module>/`.
2. Ecrire l'**entite** JPA et l'ajouter au schema via une **nouvelle** migration Flyway (`V9__...sql`), sans toucher aux migrations existantes.
3. Definir les **DTO** (record `...Request` valide, `...Response`).
4. Creer le **repository** (`extends JpaRepository`).
5. Ecrire le **service** (`@Transactional`) : regles metier, cloisonnement province, mapping vers DTO ecrit a la main.
6. Ajouter les **Specifications** si le module a une recherche filtrable.
7. Exposer le **controleur** REST (`/api/v1/...`), mince, avec `@PreAuthorize` la ou c'est necessaire.
8. Ajouter des **tests** (unitaires, et d'integration Testcontainers pour toute contrainte de base).

6.2 Cote frontend

1. Creer le dossier `features/<module>/`.
2. Definir les **modeles** TypeScript et le **service** HTTP vers `/api/v1/...`.
3. Ecrire les composants **liste**, **formulaire**, **detail**, a partir des **primitives** du design system (aucun style redefini inutilement).
4. Declarer les **routes** du module et les rattacher au menu, avec les gardes adaptees.

> Regle simple : un nouveau module doit ressembler aux modules existants. En cas de doute, copier la structure du module Vehicule (backend) et du module Vehicules (frontend), puis adapter.

7 Conventions Git et flux de travail

7.1 Branches

- `main` : code livrable, protegee.
- `develop` : integration continue, protegee.
- `feature/<module>-<description>` : une fonctionnalite.
- `fix/<module>-<description>` : une correction. `hotfix/<description>` : correction urgente sur `main`. `chore/<description>` : tache technique.

7.2 Commits (Conventional Commits)

Format : `type(scope): description a l'imperatif`. Le `scope` est le module concerne.

Type	Usage
`feat`	Nouvelle fonctionnalite.
`fix`	Correction de bug.
`refactor`	Reorganisation sans changement de comportement.
`test`	Ajout ou modification de tests.
`docs`	Documentation.
`chore, build, ci, perf`	Taches techniques, outillage, integration continue, performance.

Exemple : `feat(carte): validation d'une demande et emission de la carte.`

7.3 Pull requests

1. Rebaser sur `develop` a jour avant d'ouvrir la PR.
2. Une PR egale une tache, petite et ciblee ; remplir le modele (objectif, perimetre, test).
3. CI verte obligatoire ; un relecteur obligatoire (l'autre developpeur), pas d'auto-fusion.
4. Fusion en **squash** vers `develop`, puis suppression de la branche.

7.4 Regles de code

- Backend : injection par constructeur, `@Transactional` en couche service, aucune logique dans les controleurs.
- Frontend : composants standalone, typage strict, pas de `any`.
- Formatage verifie en CI : Spotless et Google Java Format cote backend, ESLint et Prettier cote frontend.

8 Decisions d'architecture (resume)

Les choix structurants sont consignes sous forme de decisions courtes. Resume :

Decision	Raison
Design system maison	Coherence visuelle totale, pas de librairie d'interface externe.
Mappers manuels	Mapping explicite et lisible, sans generation de code (pas de MapStruct).
Alertes in-app uniquement	Pas de service courriel valide en production ; canal courriel reporte.
Exports CSV et PDF	OpenPDF aux couleurs de la charte, un seul generateur pour tous les modules.
Tests Testcontainers	Valider sur un vrai PostgreSQL les regles portees par la base (RG3, RG4).
Init admin securisee	Aucun mot de passe par defaut : le compte admin vient de la configuration.

9 Tests et integration continue

- **Backend** : tests unitaires et tests d'integration Testcontainers (PostgreSQL reel via Docker), qui valident les migrations, le seed et les contraintes critiques RG3 et RG4.
- **Frontend** : `lint` puis `build` de production.

- **CI** (GitHub Actions) : sur chaque push et pull request vers `main` et `develop`, deux jobs (backend et frontend) doivent passer au vert avant toute fusion.

Fin du document. SRM Fleet, Oussama BENAZZOUZ et TAKI Fadlouallah.